STA 336: Statistical Machine Learning

Classwork 3

Rento Saijo

April 4, 2026

Table of Contents

Disclosure **2**

Problem 1 **3**

Problem 1 (a) 4
Problem 1 (b) 5
Problem 1 (c) 6
Problem 1 (d) 7
Problem 1 (e) 9
Problem 1 (f) 11
Problem 1 (g) 13

Disclosure

GPT-5.4 Codex was used to create the `YAML` portion and some `LaTeX` code to format the text/equations nicely. Page formatting code was also provided by Derin Gezgin. In the setup chunk, helper functions such as `table_latex()` and `predict.regsubsets()` were defined so the model-fitting output can be reported clearly and reproducibly. See the original `RMD` file here for more details.

Problem 1

Problem 1

In this group work, we predict the number of applications received using the other variables in the `College` data set.

Problem 1 (a)

Problem 1 (a)

Split the data set into a training set and a test set.

Cell 1

```r
# Load data.
college_df <- ISLR2::College

# Split data.
set.seed(24)
n         <- length(college_df$Apps)
train_idx <- sample(n, n / 2)
train_df  <- college_df[train_idx, ]
test_df   <- college_df[-train_idx, ]

# Summarize split.
split_tbl <- tibble::tibble(
  set          = c('Training', 'Test'),
  observations = c(nrow(train_df), nrow(test_df))
)

# Print split table.
split_tbl %>%
  table_latex(
    col_names = c('Set', 'Observations'),
    caption   = 'Training-test split for College.'
  )
```

Table 1: Training-test split for College.

Set	Observations
Training	388
Test	389

Using `set.seed(24)`, I split the 777 observations in `College` into 388 training observations and 389 test observations. I use this same split for all four methods so that the resulting test errors are directly comparable.

Problem 1 (b)

Problem 1 (b)

Fit a linear model using least squares on the training set, and report the test error obtained.

Cell 2

```r
# Fit least squares model.
lm_fit  <- stats::lm(Apps ~ ., data = train_df)
lm_pred <- stats::predict(lm_fit, newdata = test_df)
lm_mse  <- test_mse(test_df$Apps, lm_pred)

# Summarize least squares result.
lm_tbl <- tibble::tibble(
  model    = 'Least Squares',
  test_mse = comma_num(lm_mse)
)

# Print least squares table.
lm_tbl %>%
  table_latex(
    col_names = c('Model', 'Test MSE'),
    caption   = 'Least squares test-set performance.'
  )
```

Table 2: Least squares test-set performance.

Model	Test MSE
Least Squares	1,182,899

Fitting `Apps ~ .` on the training set gives a test MSE of $\boxed{1{,}182{,}899}$.

Problem 1 (c)

Problem 1 (c)

Fit a ridge regression model on the training set, with λ chosen by cross-validation. Report the test error obtained.

Cell 3

```r
# Create model matrices.
x_train <- stats::model.matrix(Apps ~ ., data = train_df)[, -1]
y_train <- train_df$Apps
x_test  <- stats::model.matrix(Apps ~ ., data = test_df)[, -1]
y_test  <- test_df$Apps

# Fit cross-validated ridge model.
set.seed(24)
cv_ridge <- glmnet::cv.glmnet(
  x_train,
  y_train,
  alpha  = 0,
  lambda = seq(0, 25, by = 0.1),
  nfolds = 10
)

# Predict on test data.
ridge_pred <- stats::predict(cv_ridge, newx = x_test, s = 'lambda.min')
ridge_mse  <- test_mse(y_test, ridge_pred)

# Summarize ridge result.
ridge_tbl <- tibble::tibble(
  lambda   = comma_num(cv_ridge$lambda.min, 1),
  test_mse = comma_num(ridge_mse)
)

# Print ridge table.
ridge_tbl %>%
  table_latex(
    col_names = c('$\\lambda_{\\min}$', 'Test MSE'),
    caption   = 'Ridge regression test-set performance.'
  )
```

Table 3: Ridge regression test-set performance.

$\lambda_{\min}$	Test MSE
21.4	1,255,479

Using 10-fold cross-validation on the training set, ridge regression selects $\lambda = 21.4$. The resulting test MSE is $\boxed{1{,}255{,}479}$. On this split, ridge performs slightly worse than OLS.

Problem 1 (d)

Problem 1 (d)

Fit a lasso model on the training set, with λ chosen by cross-validation. Report the test error obtained, along with the number of non-zero coefficient estimates.

Cell 4

```r
# Fit cross-validated lasso model.
set.seed(24)
cv_lasso <- glmnet::cv.glmnet(
  x_train,
  y_train,
  alpha  = 1,
  lambda = seq(0, 25, by = 0.1),
  nfolds = 10
)

# Predict on test data.
lasso_pred <- stats::predict(cv_lasso, newx = x_test, s = 'lambda.min')
lasso_mse  <- test_mse(y_test, lasso_pred)

# Extract non-zero coefficients.
lasso_coef         <- as.matrix(glmnet::coef.glmnet(cv_lasso, s = 'lambda.min'))
lasso_nonzero_idx  <- which(lasso_coef[-1, 1] != 0) + 1
lasso_nonzero_tbl  <- tibble::tibble(
  term     = rownames(lasso_coef)[lasso_nonzero_idx],
  estimate = round(lasso_coef[lasso_nonzero_idx, 1], 6)
)
lasso_summary_tbl <- tibble::tibble(
  lambda              = comma_num(cv_lasso$lambda.min, 1),
  non_zero_predictors = nrow(lasso_nonzero_tbl),
  test_mse            = comma_num(lasso_mse)
)

# Print lasso summary.
lasso_summary_tbl %>%
  table_latex(
    col_names = c('$\\lambda_{\\min}$', 'Non-Zero Predictors', 'Test MSE'),
    caption   = 'Lasso test-set performance.'
  )
```

Table 4: Lasso test-set performance.

$\lambda_{\min}$	Non-Zero Predictors	Test MSE
14.7	13	1,234,483

Cell 5

```r
# Print non-zero lasso coefficients.
lasso_nonzero_tbl %>%
  table_latex(
    col_names = c('Term', 'Estimate'),
    caption   = 'Non-zero lasso coefficients at $\\lambda_{\\min}$.'
  )
```

Table 5: Non-zero lasso coefficients at $\lambda_{\min}$.

Term	Estimate
PrivateYes	-867.672337
Accept	1.150549
Top10perc	50.904919
Top25perc	-13.593988
F.Undergrad	0.064671
P.Undergrad	0.003280
Outstate	-0.019409
Room.Board	0.221325
PhD	-5.374960
Terminal	-5.761950
perc.alumni	-6.514750
Expend	0.065172
Grad.Rate	9.303782

Lasso selects $\lambda = 14.7$. Its test MSE is $\boxed{1{,}234{,}483}$. At this value of λ, there are 13 non-zero predictor coefficients (excluding the intercept), namely `PrivateYes`, `Accept`, `Top10perc`, `Top25perc`, `F.Undergrad`, `P.Undergrad`, `Outstate`, `Room.Board`, `PhD`, `Terminal`, `perc.alumni`, `Expend`, and `Grad.Rate`.

Problem 1 (e)

Problem 1 (e)

Fit a PCR model on the training set, with M chosen by cross-validation. Report the test error obtained, along with the value of M selected by cross-validation.

Cell 6

```r
# Fit PCR with cross-validation.
set.seed(24)
pcr_fit <- pls::pcr(Apps ~ ., data = train_df, scale = TRUE, validation = 'CV')
```

Cell 7

```r
# Plot PCR validation curve.
with_family(
  pls::validationplot(
    pcr_fit,
    val.type = 'MSEP'
  )
)
```

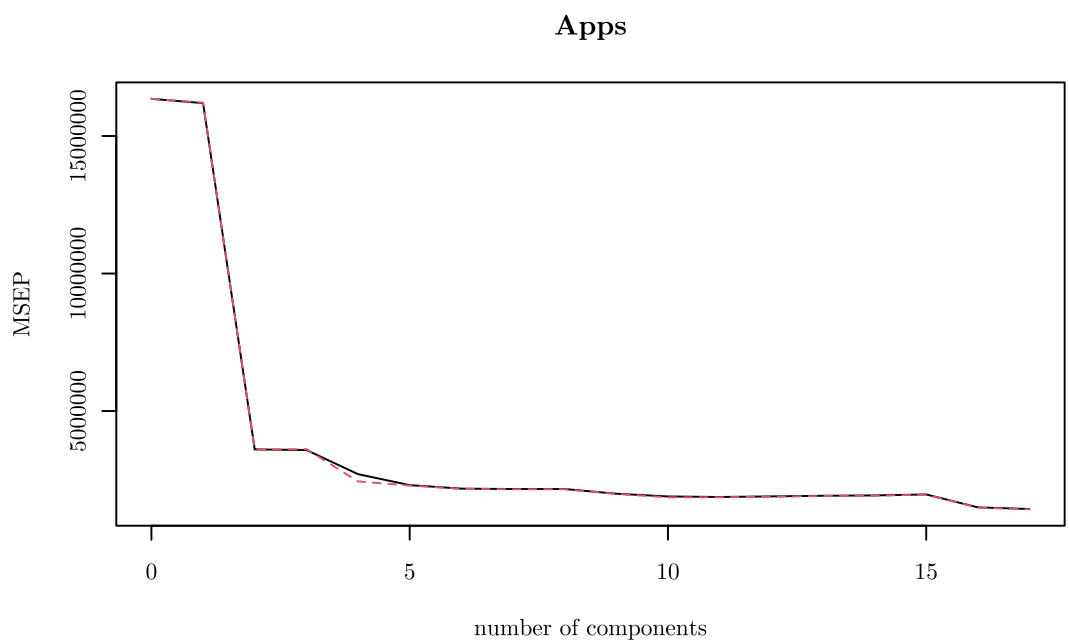

Figure 1: PCR cross-validation MSEP by number of components.

Cell 8

```r
# Select number of components by elbow.
pcr_m <- 4

# Predict on test data.
```

```r
pcr_pred <- stats::predict(pcr_fit, newdata = test_df, ncomp = pcr_m)
pcr_mse  <- test_mse(y_test, pcr_pred)

# Summarize PCR result.
pcr_tbl <- tibble::tibble(
  selected_m = pcr_m,
  test_mse   = comma_num(pcr_mse)
)

# Print PCR table.
pcr_tbl %>%
  table_latex(
    col_names = c('Selected $M$', 'Test MSE'),
    caption   = 'PCR test-set performance.'
  )
```

Table 6: PCR test-set performance.

Selected M	Test MSE
4	3,371,475

From the cross-validation MSEP plot, the curve drops very sharply at first and still decreases substantially through $M = 4$, but after that the gains become much smaller. In particular, the CV MSEP falls from 3,601,660 at $M = 2$ to 2,445,960 at $M = 4$, whereas the later decreases are comparatively modest. Based on that elbow, I choose $\boxed{M = 4}$. With 4 principal components, the resulting test MSE is $\boxed{3{,}371{,}475}$.

Problem 1 (f)

Problem 1 (f)

Comment on the results obtained. How accurately can we predict the number of college applications received? Is there much difference among the test errors resulting from these four approaches?

Cell 9

```r
# Create comparison table.
comparison_tbl <- tibble::tibble(
  model    = c('Least Squares', 'Ridge', 'Lasso', 'PCR'),
  tuning   = c(
    'None',
    paste0('$\\lambda = ', comma_num(cv_ridge$lambda.min, 1), '$'),
    paste0('$\\lambda = ', comma_num(cv_lasso$lambda.min, 1), '$'),
    paste0('$M = ', pcr_m, '$')
  ),
  test_mse = c(
    comma_num(lm_mse),
    comma_num(ridge_mse),
    comma_num(lasso_mse),
    comma_num(pcr_mse)
  )
)

# Print comparison table.
comparison_tbl %>%
  table_latex(
    col_names = c('Model', 'Tuning', 'Test MSE'),
    caption   = 'Comparison of the four required methods.',
    escape    = FALSE
  )
```

Table 7: Comparison of the four required methods.

Model	Tuning	Test MSE
Least Squares	None	1,182,899
Ridge	$\lambda = 21.4$	1,255,479
Lasso	$\lambda = 14.7$	1,234,483
PCR	$M = 4$	3,371,475

Among the four required approaches, least squares performs best with test MSE 1,182,899. Lasso is close behind at 1,234,483, ridge is slightly worse at 1,255,479, and PCR is much worse at 3,371,475 when M is chosen by the elbow method. The gap between the best and worst methods is 2,188,576, so here there is a substantial difference. The main conclusion is that least squares, ridge, and lasso all perform in the same general range on this split, while PCR loses too much predictive information when we stop at the elbow rather than using all

components. So for this data set, the direct shrinkage methods are much more competitive than PCR under the revised component-selection rule.

Problem 1 (g)

Problem 1 (g)

If you have more time, what else can you try to build a model for predicting the number of applications received? Can it improve the models you have built?

Cell 10

```r
# Fit best subset models.
best_subset_fit <- leaps::regsubsets(Apps ~ ., data = train_df, nvmax = 17)
best_subset_mse <- sapply(1:17, function(i) {
  pred <- predict.regsubsets(best_subset_fit, test_df, id = i)
  test_mse(y_test, pred)
})

# Extract best subset result.
best_subset_size <- which.min(best_subset_mse)
best_subset_tbl  <- tibble::tibble(
  model_size = best_subset_size,
  predictors = paste(names(stats::coef(best_subset_fit, id =
  best_subset_size))[-1], collapse = ', '),
  test_mse   = comma_num(min(best_subset_mse))
)

# Print best subset extension.
best_subset_tbl %>%
  table_latex(
    col_names  = c('Model Size', 'Predictors', 'Test MSE'),
    caption    = 'Best subset extension on the same split.',
    align      = c('c', 'l', 'c'),
    wrap_col   = 2,
    wrap_width = '8cm'
  )
```

Table 8: Best subset extension on the same split.

Model Size	Predictors	Test MSE
1	Accept	1,093,185

One natural extension, since we used it in class, is best subset selection with `leaps::regsubsets()`. On this same split, the best subset approach actually improves on the four required models: the lowest test error comes from the 1-variable model using only `Accept`, and its test MSE is $\boxed{1{,}093{,}185}$. This is 89,714 lower than the best test MSE among the required four methods, so in this case, yes, trying an additional model-selection approach does improve predictive performance.